PawGuard Backend — System Architecture

Whole-Backend Overview — pawguard-backend (single source of truth)

PawGuard Backend | FastAPI 3.13 + PostgreSQL 17 + Redis/ARQ + S3 | Generated from repo @ commit 99a8430 (2026-08-12)

---

1. What This System Is

A single “modular monolith” FastAPI backend that is the system of record for the entire PawGuard ecosystem: the public website (Next.js), the staff/admin portal (React), and three Flutter apps (Public, Staff, Executive/Admin). It follows Domain-Driven Design and Clean Architecture, with 23 domain modules covering rescue operations, shelter/medical care, adoption, fostering, volunteers, donations/finance, and the owner-facing pet-safety features (companion pets, lost & found, QR tags).

2. Technology Stack

Layer	Technology
Runtime	Python 3.13
Framework	FastAPI
ORM	SQLAlchemy 2.x (async)
Database	PostgreSQL 17 (Supabase-hosted in prod)
Migrations	Alembic
Cache / Queue	Redis + ARQ
Validation	Pydantic v2
Auth	JWT (RS256) + Argon2id password hashing
MFA	TOTP (pyotp)
File storage	AWS S3 (presigned URLs)
Background jobs	ARQ worker (cron + queued jobs)
Logging	structlog (structured JSON logs)
Containerisation	Docker; package manager uv

3. Request Flow — Layered Architecture

Client → Router → Service → Repository → Database

- **Routers** — authenticate, authorize (RBAC/PBAC permission codes), validate input, call services, shape the response.
- **Services** — own 100% of business behaviour and domain logic; the only layer allowed to make business decisions.
- **Repositories** — data access only, no business logic.
- **Core** — cross-cutting concerns: config, structured logging, security, exception handling, pagination, rate limiting.

This separation is enforced project-wide by AGENTS.md, the repository's “engineering constitution” — business logic is never permitted to live in a router or repository.

4. Codebase Footprint (this snapshot)

Metric	Value
Domain modules	23 (+ 1 runtime output directory: generated_reports/)
Total API endpoints (auto-counted)	~431
Total owned DB tables (auto-counted)	~81
Total git commits	153+ (README) — latest: 99a8430, 2026-08-12

5. All 23 Modules

#	Module	One-line Purpose
1	auth	Identity, sessions, RBAC/PBAC, MFA, OAuth, audit log
2	admin	Cross-module audit viewing + admin analytics dashboard
3	dashboards	Shared aggregation layer for role-specific dashboard views
4	rescue	Rescue intake, field triage, dispatch tracking
5	rescue_centre	Rescue-centre / facility directory (built on shelter model)
6	dog	Shelter-admitted dog master profiles, medical/behavior, media
7	companion_pet	Owner pets, QR safety tags, clinics, appointments, reminders
8	adoption	Adoption applications, approvals, contracts, follow-ups
9	volunteer	Volunteer registration, scheduling, hours, training
10	foster	Foster applications, placements, returns
11	donation	Donor management, pledges, receipts
12	lost_found	Lost/found pet matching, broadcast alerts, ownership claims
13	inventory	Stock levels, movement tracking, reorder triggers
14	shelter	Facility capacity, room/kennel assignment, status
15	medical	Shelter-side medical records: treatments, vaccinations, surgery
16	portal	Public website content + public submission endpoints (largest module)
17	fleet	Rescue-transport vehicles, maintenance, trips, fuel
18	grievance	Complaint/grievance intake and resolution
19	notifications	Shared in-app/email/push delivery layer
20	settings	Global settings and feature flags
21	storage	Presigned S3 upload issuance + confirmation
22	finance	Ledger reconciliation across donations/expenses
23	reports	On-demand CSV/XLSX/PDF report generation

6. Cross-Cutting Infrastructure

- **Background workers (ARQ):** email delivery, push/in-app notifications, report generation, image processing, companion-pet reminders (daily cron 09:45), lost-pet broadcast fan-out.
- **Rate limiting:** Redis sliding-window, applied per-endpoint (e.g. lost-pet broadcast 3/hour, QR scan 20/60s).
- **File uploads:** universal presigned-S3 pattern via the storage module — files never transit the API server.
- **Audit trail:** AuthAuditEventType events recorded centrally and surfaced through the admin module.
- **Health endpoints:** GET /health, /live, /ready (liveness + DB/Redis dependency checks).

7. Deployment Shape

Docker image built from a single Dockerfile; docker-compose.yml for local Postgres+Redis, docker-compose.prod.yml for production (API + ARQ worker + Redis, pointed at a remote/Supabase DB). On Render's free tier, pawguard.serve runs the API and the ARQ worker as concurrent asyncio tasks in one process so no paid Background Worker service is required; if Redis is unreachable the API still serves requests and jobs degrade to a no-op rather than crashing the service.

8. Related Documents in This Set

Each of the 23 modules above has its own 1–2 page architecture document (data model, endpoints, permissions, key service operations) generated alongside this overview. The companion_pet and lost_found modules additionally have a

deeper, feature-level document set (System Architecture, API Spec, DB Schema, QR engine, broadcast/GPS/scan flows, vet directory, appointments, reminders) produced in an earlier review round.

Source: repo README.md, src/pawguard/api/v1/router.py (module wiring), AGENTS.md, and auto-extraction across all modules//models.py, router*.py, service.py at commit 99a8430 (2026-08-12).*